Relatório — CardioIA Fase 3, Parte 1

Relatório — Parte 1: Armazenamento e processamento local (Edge Computing)

CardioIA — Fase 3 · Victor Copque dos Reis (RM566821) · Victor Hugo Ferreira Rolim (RM568006)

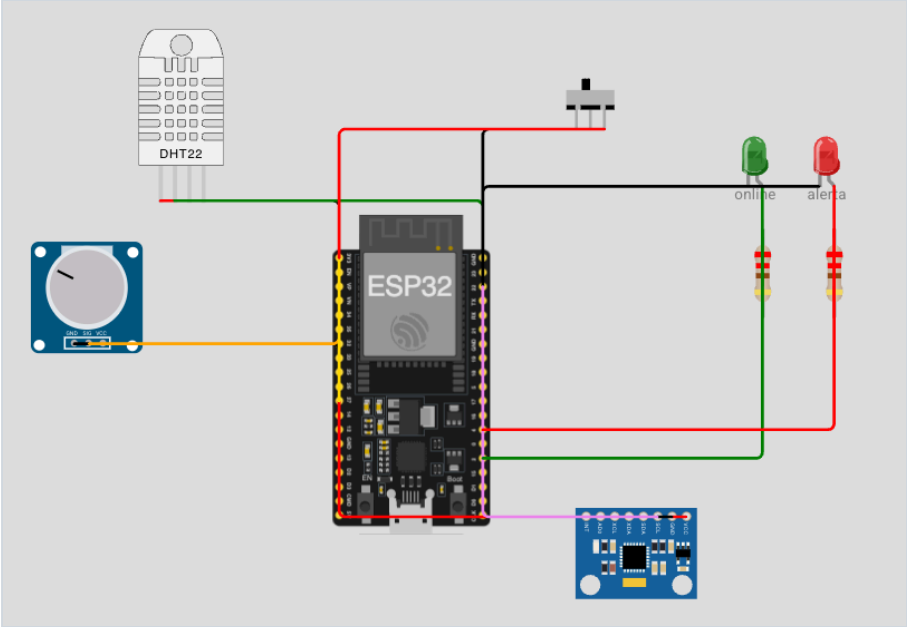
Projeto no Wokwi: https://wokwi.com/projects/479413654575407105

1. O que o protótipo simula

Um vestível de monitoramento cardíaco: uma pulseira ou patch que acompanha o paciente fora do hospital, registra sinais vitais o tempo todo e entrega os dados à equipe de saúde quando há conexão. O protótipo roda num ESP32 no Wokwi com três sensores:

Sensor	Grandeza	Por que está no vestível
DHT22 (obrigatório)	temperatura (°C) e umidade (%)	febre e hipotermia; a umidade da pele é um indício de sudorese
Potenciômetro como sensor de pulso	frequência cardíaca (BPM)	o sinal central de um monitor cardíaco. O Wokwi não tem sensor de pulso, então o potenciômetro representa a saída já processada de um MAX30102
MPU6050 (acelerômetro)	aceleração em g	diz se o paciente está em repouso ou em atividade e detecta queda

Uma chave deslizante faz o papel da variável booleana `wifiConectado`. O LED verde indica que o dispositivo está online e o vermelho acende quando a última leitura gerou alerta.



Circuito no Wokwi: ESP32, DHT22, potenciômetro (pulso), MPU6050, chave de Wi-Fi e LEDs de status

2. Fluxo de funcionamento



1. **Captura.** A cada 10 s em produção (2 s no modo demonstração) o `loop()` lê os três sensores. A coleta nunca depende do estado da conexão.
2. **Processamento no Edge.** Antes de gravar, o próprio ESP32 interpreta a leitura:
   - classifica o movimento pelo módulo da aceleração: perto de 1 g é **repouso**, desvio acima de 0,3 g é **atividade** e pico acima de 2,5 g é **queda**;
   - aplica as regras de alerta: bradicardia (< 50 BPM), taquicardia, febre (≥ 37,8 °C), hipotermia (≤ 35 °C), queda e falha do DHT22.

   A taquicardia depende do contexto: o limite é 120 BPM em repouso e 150 BPM em atividade. 140 BPM correndo é esperado, 140 BPM deitado não é. Decidir isso no dispositivo evita falso alarme e não exige nuvem para disparar o LED de alerta.
3. **Armazenamento.** Toda leitura, com ou sem conexão, é acrescentada ao arquivo `/leituras.csv` no SPIFFS, no formato `seq,t_s,temp_c,umid_pct,bpm,acel_g,movimento,alerta`. Passar sempre pelo buffer (*store-and-forward*) deixa um único caminho de envio, testado o tempo todo, em vez de um caminho "online" e outro "offline".
4. **Transmissão.** Com `wifiConectado = true`, o buffer é lido linha a linha e cada linha é enviada com `Serial.println` (prefixo `[NUVEM]`). Ao final o arquivo local é apagado. Na Parte 2 esse envio passa a ser MQTT.

3. Lógica de resiliência offline

Coleta contínua. Sem conexão, o sistema segue lendo, processando e gravando. O log mostra `[OFFLINE] Leitura #N gravada no SPIFFS (x/limite)`.

Sincronização ao reconectar. Quando a chave volta para "conectado", tudo que se acumulou é enviado em ordem, antes das leituras novas.

Queda no meio da sincronização. O Wi-Fi é checado antes de cada linha. Se cair durante o envio, as linhas que ainda não saíram são regravadas num arquivo novo, que substitui o anterior. Nada se perde e nada é enviado duas vezes. O arquivo só é apagado quando o envio termina por completo.

Número de série (`seq`). Cada leitura tem um número sequencial. Se um lote chegar repetido à nuvem (por exemplo, se o dispositivo reiniciar durante um envio), o servidor deduplica por `seq`.

Retomada após reinício. No boot o código conta as linhas pendentes no arquivo. Num ESP32 real, que mantém o SPIFFS sem energia, o dispositivo reinicia sabendo o que ainda falta enviar.

4. Estratégia de armazenamento limitado

Dimensionamento pelo modelo de negócio. O caso de uso é um paciente que passa a noite e parte do dia seguinte longe do celular ou gateway. Definimos **24 horas de autonomia offline**:

Parâmetro	Produção	Demonstração
Intervalo entre leituras	10 s	2 s
Capacidade do buffer	**8.640 registros** (24 h)	20 registros
Espaço ocupado	~45 bytes/linha → ~390 KB	< 1 KB

Os ~390 KB cabem com folga na partição SPIFFS padrão do ESP32 (~1,4 MB). O modo demonstração (`MODO_DEMO`) existe só para que a compactação apareça em poucos segundos de simulação.

O que acontece quando enche. Descartar simplesmente as leituras mais antigas seria errado num monitor cardíaco: o episódio de taquicardia de madrugada é justamente o que o médico precisa ver. A compactação faz duas passadas:

1. **Decimação:** mantém **todas as leituras com alerta** e metade das normais (uma sim, uma não). A série normal perde resolução, mas continua cobrindo o período inteiro.
2. **Descarte das mais antigas**, só se a decimação não bastou para voltar a 75% da capacidade, por exemplo depois de horas seguidas em alerta. Esse descarte fica registrado no log.

Compactar até 75%, e não até 99%, evita que a compactação rode de novo a cada leitura.

5. Limitações do simulador

Como o enunciado aponta, o SPIFFS do Wokwi funciona **durante a sessão** (dá para ver o arquivo crescer, ser compactado e ser apagado), mas some quando a simulação é reiniciada. Para funcionar no Wokwi, o projeto inclui um `partitions.csv` com uma partição `spiffs` de 1,375 MB: sem ela, o ESP32 simulado não tem área de dados e a montagem falha. Num ESP32 físico o mesmo código mantém os dados entre quedas de energia. Por isso o Monitor Serial também funciona como trilha de auditoria: cada leitura, gravação, compactação e envio aparece nele.

Outras simplificações: o pulso vem de um potenciômetro, não de um sensor óptico; o tempo é contado desde o boot (`millis()`), e um dispositivo real sincronizaria o relógio por NTP ao se conectar; e o "envio à nuvem" é a porta serial.

6. Uso responsável

O protótipo é acadêmico e todos os dados são simulados. Ele sinaliza situações para revisão humana e não diagnostica. Num produto real, os dados de saúde exigiriam criptografia em repouso no SPIFFS e TLS no transporte (LGPD, art. 11).